Android programmering eksamen rapport

PGR 112

Gruppenummer 4

Kandidatnummer: 29, 28, 20

Innledning 2

Skjerm 1: Alle Anime 3

Skjerm 2: Søke etter Anime ID 5

Skjerm 3: Egne Anime Ideer 7

Skjerm 4: Favoritt Anime liste 9

Del 2: Redegjørelse av planlegging og utvikling 11

- Planleggingsfasen 11
- Arbeidsflyt og fordeling av oppgaver 11
- Avgjørelser 11
- Samarbeid 12

Litteraturliste 12

- Etisk bruk av KI 13

Innledning

Denne applikasjonen er utviklet for å lage en komplett løsning basert på moderne Android teknologi med utgangspunkt i et JIKAN-API som leverer oppdatert anime-data. Appen lar brukeren utforske et utvalg av anime-serier, søke etter enkeltserier ved hjelp av ID, registrere egne anime ideer og administrere en personlig favorittliste. Gjennom hele arbeidet har vi hatt fokus på å kombinere teknisk korrekt implementasjon med en brukervennlig og tydelig struktur.

For å bygge løsningen har vi brukt hovedteknikkene fra emnet: Jetpack Compose for UI, Retrofit for API-kommunikasjon, Room for lokal lagring, Compose Navigation for flyt mellom skjermene og MVVM-arkitektur for å skille logikk og presentasjon (*Android Developers, 2025 – Guide to app architecture; PGR208 - Forelesning 6: App Architecture & ViewModel*). I tillegg har vi brukt Coil til bildehåndtering *(Android Developers, 2025 – Loading Images, 2024)*. Selv om fargetilpasning og tema ikke har vært en del av undervisningen, valgte vi å undersøke dette på egen hånd og implementere et enklere fargeoppsett inspirert i anime-estetikk. Dette gjorde vi for å skape en mer engasjerende visuell opplevelse og samtidig lære mer om MaterialTheme i Compose *(Android Developers, 2025 – Material Design 3 in Compose)*

Appen består av fire hovedskjermer som dekker alle kravene i oppgaven: visning av alle animeer, søk etter anime-ID, en skjerm for brukerens egne ideer og en egendefinert skjerm, samt ekstrafunksjonaliteter. Målet har vært å utvikle en løsning som både viser forståelse for teknikkene vi har lært i emnet, og som gir en helhetlig og intuitiv brukeropplevelse.

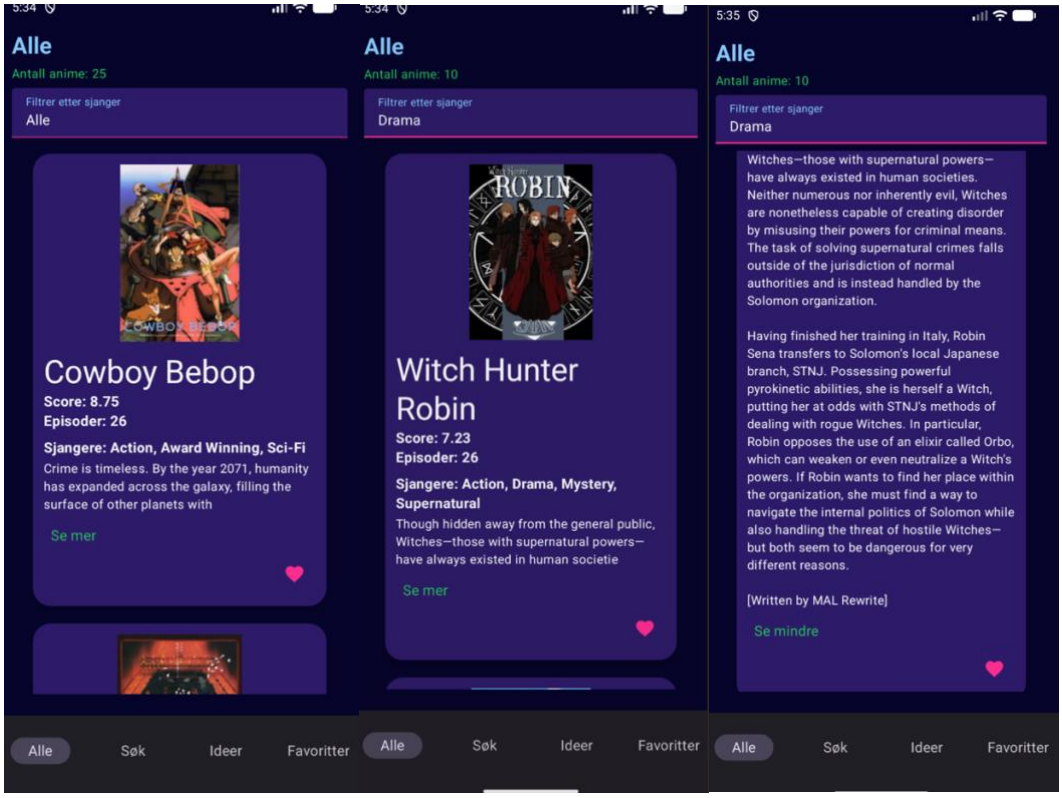
Skjerm 1: Alle Anime

Skjerm 1 utgjør hovedoversikten til applikasjonen vår, og presenterer en dynamisk liste med anime hentet fra JIKAN-API-et. Skjermen demonstrerer sentrale teknikker i Compose, som reaktiv tilstandshåndtering, filtrering og dynamisk listevisning.

Data lastes inn via AnimeViewModel, som igjen benytter AnimeRepository til å utføre API kall gjennom Retrofit. ViewModelen eksponerer resultatet som en StateFlow, og UI-et observerer dette med collectAsState *() (Android Developers, 2025 – State and Jetpack Compose; PGR208 - Forelesning 6 App Architecture & ViewModel*). Dermed oppdateres listen automatisk når API-responsen kommer inn, uten behov for manuell oppdatering. Denne tilnærmingen gir en mer responsiv og forutsigbar brukeropplevelse.

Selve listevisningen er gjennomført med LazyColumn, som gjør at appen kan håndtere store mengder data på en effektiv måte. *(PGR208 - Forelesning 4: Navigation & Lifecycle)* Hver anime presenteres i et Card med bilde (lastet via Coil), tittel, sjanger, score og en kort beskrivelse hentet fra API-et. For å unngå at lange beskrivelser dominerer layouten, har hvert kort mulighet for ekspandering gjennom en lokal expandedCard-state *(PGR208 - Forelesning 2-3: Compose UI)* «Se mer/Se mindre» lar brukeren kontrollere informasjonsmengden på en ryddig måte.

Hvert kort inneholder også en favorittknapp, som lagrer valgt anime i Room-databasen gjennom FavoritesAnimeViewModel. Ikonet markerer interaksjon visuelt og gir umiddelbar bekreftelse til brukeren om at handlingen er utført. Fargevalg og typografi hentes fra applikasjonens tilpassede MaterialTheme for å sikre et konsistent uttrykk.



Skjermdump 1 *Skjermdump 2* *Skjermdump 3*

Skjermdump 1 – Normaltilstand

Viser standardtilstanden der ingen filtrering er aktiv. Listen er dynamisk og inneholder alle anime som er hentet fra API-et. Antall anime vises øverst og oppdateres automatisk basert på antall elementer i StateFlow. Hvert kort presenterer bilde, metadata og en kort beskrivelse med en «se mer» knapp.

Skjermdump 2 – Filtrering aktivert

Viser listen filtrert etter sjangeren drama, og antall anime reduseres fra 25 til 10 basert på valgt filter. Dropdown menyen viser gjeldende valg, og LazyColumn viser kun elementer som matcher valgt sjanger. UI-et oppdateres automatisk uten at skjermen lastes inn på nytt, noe som viser fordelene med Compose sin tilstandshåndtering.

Skjermdump 3 – Utvidet visning

Viser hvordan et kort ser ut når brukeren trykker «Se mer». Hele beskrivelse teksten vises, samtidig som knappen endres til «se mindre». Utvidelsen gjelder kun det valgte kortet, ettersom hver rad har sin egen expandedCard-state. Dette gir god kontroll over listevisningen når beskrivelsene er lange.

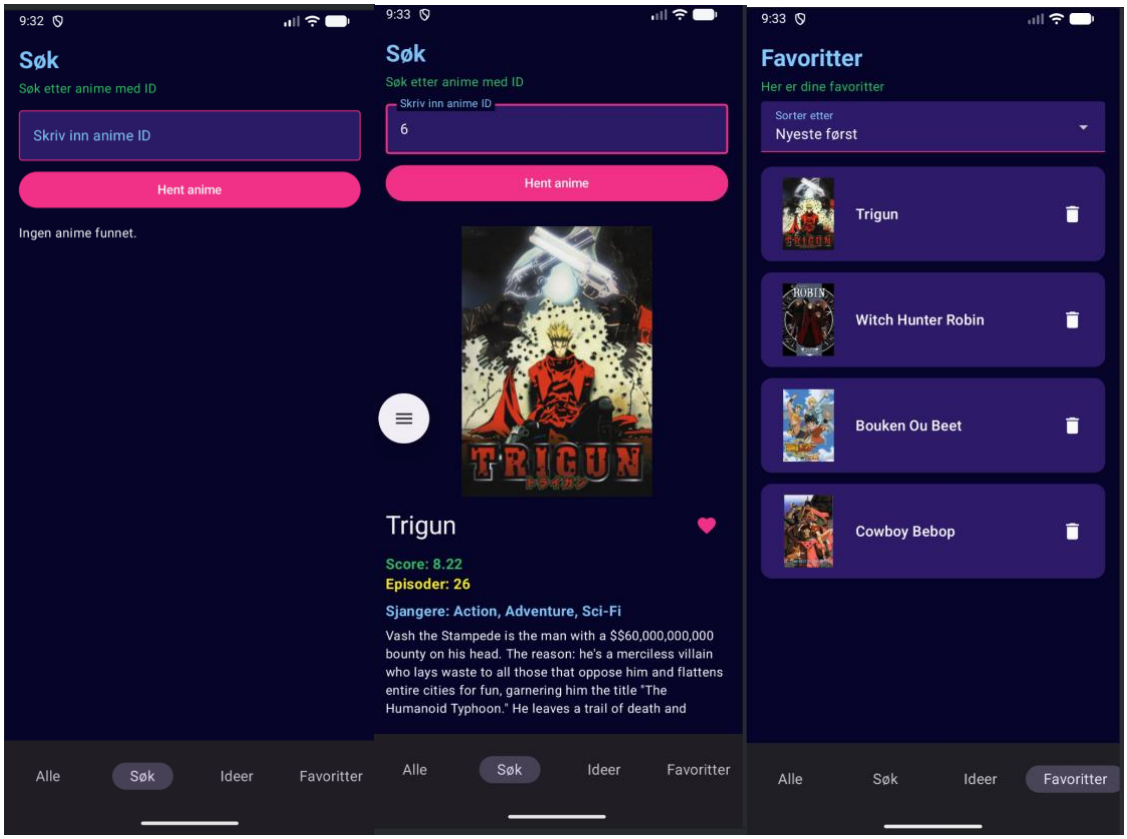
Skjerm 2: Søke etter Anime ID

Skjerm 2 lar brukeren søke etter en spesifikk anime basert på id, og gir brukeren detaljer som bilde, tittel, rating, episodetall, sjanger og beskrivelse. Brukeren får rask tilgang til informasjon og mulighet til å legge til anime i favoritter som en ekstra funksjonalitet. Skjermen bruker både API-kall via Retrofit og lokal database i Room så favoritt-funksjonen fungerer uavhengig av internettforbindelse. *(PGR208 - Forelesning 8-9: HTTP & Retrofit)* Grunnen til at vi har gjort dette er for å gjøre appen så brukervennlig og robust som mulig ved at den kan brukes offline i tillegg til online.

Når brukeren søker på en Id og trykker på søk knappen sendes forespørsel til skjermens ViewModel som håndterer API-kallet og state via mutableStateOf og collectAsState (). Dette gjør at UI oppdaterer seg automatisk når dataene om bilde, tittel, rating, antall episoder, sjanger og beskrivelse av Anime er hentet.

Strukturen følger Material3 sine anbefalinger på UI-struktur. Vi bruker Column for hovedlayout for en naturlig rekkefølge og flyt i innholdet. Innholdet man må scrolle for å se plasseres i en LazyColumn der hver anime ligger i hver sin Card. Tittelen og favoritt knappen ligger sammen i en row for at de skal ligge på samme linje horisontalt og at det skal være tydelig for bruker akkurat hvilken anime som legges til i favoritter da knappen er ved siden av tittelen. Vi bruker også Coil for å laste inn bilder.

Brukerhandlingene på skjermen inkluderer: å søke etter spesifikk anime basert på id, scrolle for å lese mer om anime, legge til anime i favoritter og navigere til de andre skjermene via navigasjonsbaren. Brukeren får opp feilmelding ved feil id og bekreftelse på at anime blir lagt til i favoritter ved at favoritt knappen får en lysere skygge rundt ikonet når den blir trykket på.



Skjermdump 1 *Skjermdump 2* *Skjermdump 3*

Skjermdump 1 - Normaltilstand:

Viser skjermens normaltilstand før brukeren har gjort et søk. Bruker Compose-komponentene TextField og Button for bruker input. State håndteres med remember og mutableStateOf så UI oppdateres når bruker skriver. Bruker får også tilbakemelding om det ikke er anime med samme id som den har skrevet inn.

Skjermdump 2 - Søk:

Viser data om anime fra ViewModel. Data gis som StateFlow som UI leser via collectAsState, sånn at skjermen oppdaterer seg automatisk etter API-kall. Bildet lastes opp med Coil og anime informasjonen bygges med Card og LazyColumn for struktur.

Skjermdump 3 - Lagt til i favoritter:

Viser at anime blir lagt til i favoritter når man trykker på favoritt-knappen.

Skjerm 3: Egne Anime Ideer

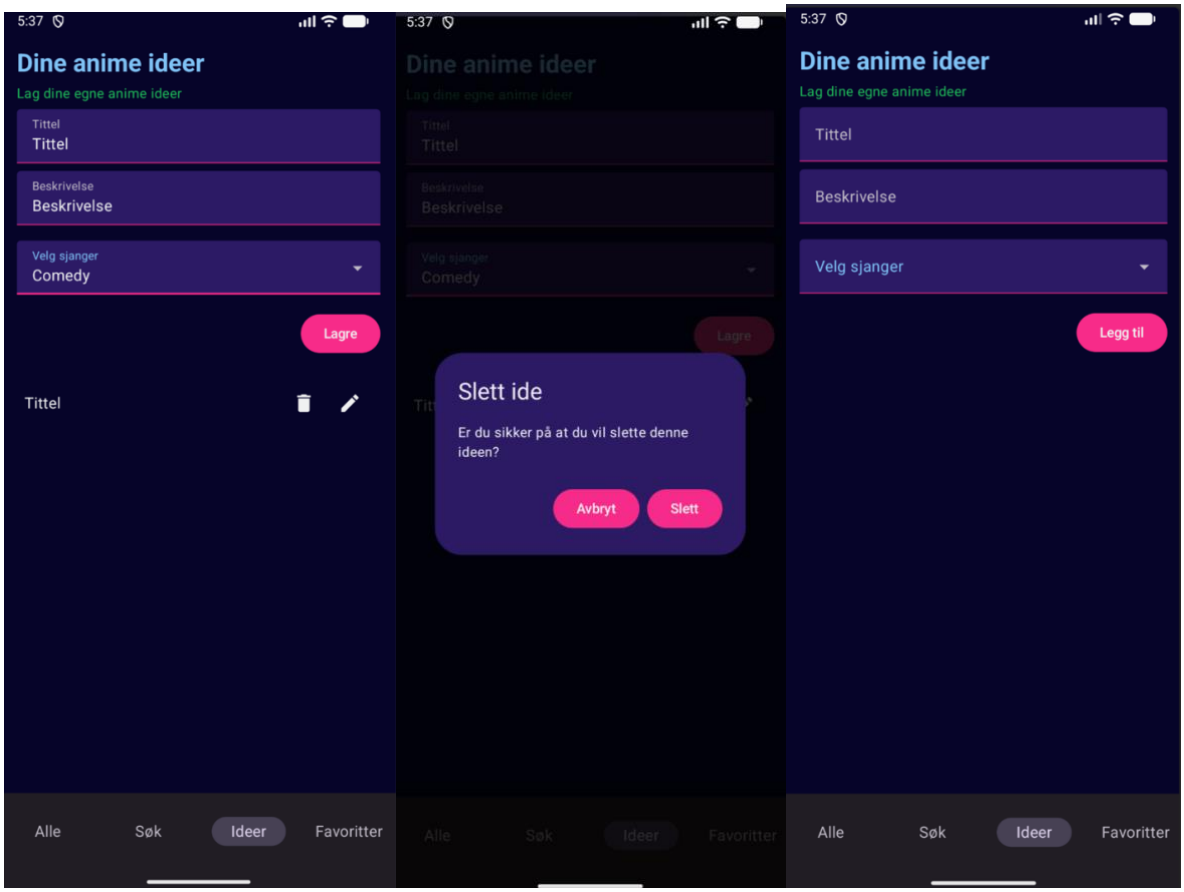
Skjerm 3 gir brukeren muligheten til å opprette, lagre, vise, redigere og slette egne anime-ideer ved hjelp av lokal lagring gjennom Room. Dataene lagres permanent og hentes som en reaktiv Flow, slik at UI-et alltid er synkronisert med databasen (*PGR208 - Forelesning 11: Local Storage Extanded)*. Dette gir skjermen full CRUD-funksjonalitet.

Implementasjonen følger MVVM-arkitekturen. AnimeIdeaViewModel mottar et DAO-interface via en Factory, og eksponerer listen som en StateFlow *(PGR208 - Forelesning 6: App Architecture & ViewModel)*. Ved å observere dette med collectAsState () oppdateres UI-et automatisk når brukeren legger til, redigerer eller sletter ideer. Dette valget gir en tydelig ansvarfordeling: all logikk ligger i ViewModelen, mens skjermen kun håndterer visning og brukerinteraksjon.

UI-et består av tre inputfelt (tittel, beskrivelse og sjanger), der sjanger håndteres med ExposedDropdownMenuBox. Brukeren kan enten legge til en ny ide eller redigere en eksisterende, avhengig av om editIdea er satt. Etter lagring nullstilles hele skjemaet gjennom Compose-state, noe som gir god fly for brukeren.

Ideene listen i en LazyColumn ved hjelp av en separat, gjenbrukbar komponent: AnimeIdeaItem. Denne komponenten inneholder tittel, sletteknapp og redigeringsknapp. onDelete kaller både sletting i databasen og onDeleteSelected slik at UI-er ryddes hvis brukeren var i detaljvisning på samme ide. Sletting bekreftes via en AlertDialog, som følger anbefalt praksis for destruktive handlinger. Ved trykk på en rad vises ideens detaljer, uten navigasjon til ny side.

Kombinasjonen av Room, Flow, MVVM, komponentbasert UI, og full CRUD viser god forståelse av pensum. Skjermen er både teknisk robust og brukervennlig, og oppfyller tilleggskravene til interaktivitet og økt kompleksitet.



Skjermdump 1 *Skjermdump 2* *Skjermdump 3*

Skjermdump 1 - Normalvisning:

Viser skjemaet når brukeren har lagt til ide og trykket rediger knappen. Material3-komponenter og LazyColumn strukturerer skjermen klart og forutsigbart.

Skjermdump 2 – Slettedialog:

AlertDialog brukes for trygghet ved sletting. State-styring gjør dialogen stabil og forutsigbar.

Skjermdump 3 - Oppdatert liste:

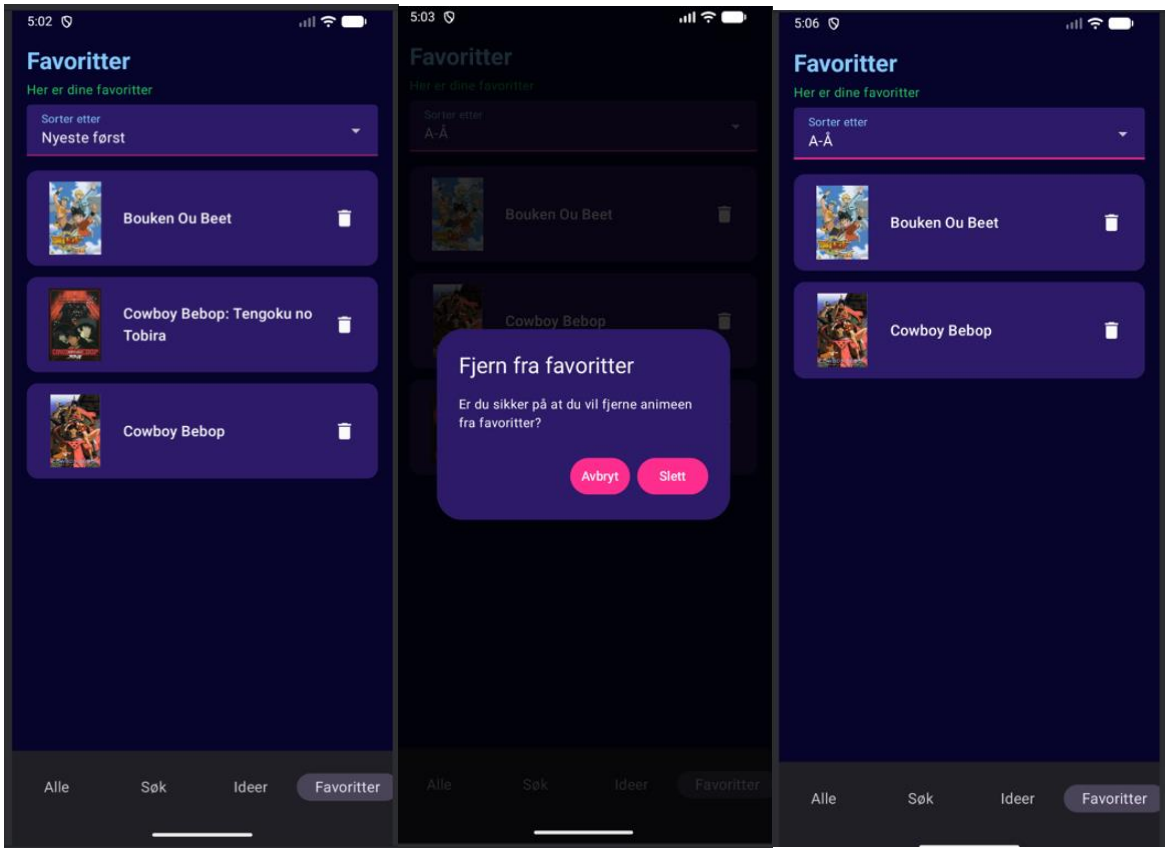
Etter sletting nullstilles input via Compose-state. Dette viser reaktiv databehandling og ren MVVM-struktur.

Skjerm 4: Favoritt Anime liste

Skjerm 4 er applikasjonen egendefinerte skjerm, der vi valgte å implementere et favorittsystem. Denne funksjonaliteten gir brukeren et personlig område i appen, og vanlig i moderne innholdsbaserte apper, noe som gjør den både nyttig og tematisk relevant. Skjermen er bygget med Jetpack Compose og følger MVVM-arkitektur *(PGR208 - Forelesning 6: App Architecture & ViewModel*). FavoritesAnimeViewModel henter favoritter fra Room-databasen gjennom en Flow som observeres i UI med collectAsState () *(PGR208 - Forelesning 11: Local Storage Extended).* Dette gjør at brukerens favoritter oppdateres automatisk når de legges til, slettes eller sorteres, uten behov for manuelle oppdateringer. Dette valget ble tatt for å sikre et reaktivt og stabilt grensesnitt der endringer alltid gjenspeiles umiddelbart uten ekstra logikk i UI-laget.

Sorteringensfunskjonaliteten er implementert gjennom en enum class i ViewModel. Dette gir en ren og skalerbar løsning, og ved å legge ansvaret i ViewModel beholder vi et tydelig skille mellom logikk og presentasjon. Til sorteringsfeltet valgte vi ExposedDropdownMenuBox fra Material 3, både fordi den gir riktig input-adferd og fordi den følger Android designretningslinjer med tydelig label, ikon og fokusindikator *(PGR208 - Forelesning 2/3: Compose UI).* Listevisningen bruker en LazyColumn, fordi den håndterer dynamiske lister effektivt. Hvert anime-element vises i et Card, valgt for å gi en tydelig visuell struktur og et gjenkjennbart hierarki uten å overbelaste skjermen. Vi valgte bevisst å vise kun bilde og tittel for å holde skjermen ren og effektiv, vi vurderte å vise flere detaljer, men valgte bort dette av hensyn til brukeropplevelse og kompleksitet.

Ved sletting får brukeren en AlertDialog, dette er anbefalt ved destruktive handlinger og gir sikkerhet ved å kreve bekreftelse. Etter sletting oppdateres listen umiddelbart gjennom Room + Flow + Compose-recomposition, som fungerer som en naturlig «OK»-tilbakemelding uten ekstra meldinger.



Skjermdump 1 *Skjermdump 2* *Skjermdump 3*

Skjermdump 1 - Normalvisning:

LazyColumn og Card ble valgt for effektiv listehåndtering og tydelig visuell struktur. Coil brukes for asynkron bildeflyt, noe som gjør at skjermen holder seg responsiv.

Skjermdump 2 – Slettedialog:

AlertDialog gir nødvendig trygghet før destruktiv handling. State-styringen med remember {} gjør visningen forutsigbar og enkel å holde ren.

Skjermdump 3 - Oppdatert liste:

Flow + StateFlow gir automatisk UI-oppdateringer. Dette viser hvorfor vi valgte MVVM, UI-et håndterer ingen logikk, men reagerer kun på dataendringer.

Del 2: Redegjørelse av planlegging og utvikling

Planleggingsfasen

Vi startet arbeidet med en detaljert gjennomgang av oppgaveteksten for å tydelig definere funksjonelle krav og forstå vurderingskriteriene. Basert på dette identifiserte vi de fire obligatoriske skjermene og avklarte hvilken informasjon hver skjerm måtte hente, lagre eller presentere for brukeren. Dette dannet grunnlaget for en overordnet arkitektur basert på MVVM, for en ryddig struktur og tydelig skille mellom logikk og UI *(PGR208 - Forelesning 6: App Architecture & ViewModel).* Vi tenkte også på en felles Repository som mellomlag mellom databasen, API-et og ViewModelene for å sikre konsistent og skalerbar databehandling *(PGR208 - Forelesning 10/11: Local Storage Exptended).* I planleggingsfasen tok vi også stilling til tilleggskravene, hvor vi valgte å utvide appens funksjonalitet med filtrering og et favorittsystem for å gi brukeren mer interaksjon, i tråd med oppgavens intensjon.

Arbeidsflyt og fordeling av oppgaver

Vi jobbet i et felles GitHub-repository for å sikre ryddig versjonskontroll og kontinuerlig deling av fremdrift. Selv om vi samarbeidet tett, fikk hver person et naturlig hovedfokus: en jobbet mest med API og databehandling i ViewModel, en tok ansvar for Room og de lokale skjermene, og en jobbet mest med UI-struktur og navigasjon. Samtidig tok vi oppgaver på tvers når det var nødvendig, slik at ingen deler av prosjektet ble avhengig av en person.

Avgjørelser

Avgjørelser ble tatt løpende basert på teknisk krav, beste praksis fra undervisningen og hva som ga mest mening for brukeropplevelsen. Ved utfordringer diskuterte vi alternativer sammen, analyserte feillogger og sammenlignet løsninger før vi gjorde endringer. Målet var alltid en ren struktur, stabil funksjonalitet og konsistent bruk av MVVM.

Samarbeid

Samarbeidet fungerte godt, både digitalt og fysisk. Vi kommuniserte kontinuerlig, hjalp hverandre med feilsøking og delte ansvar når noe stoppet opp. Dette førte til jevn fremdrift og ga alle innsikt i hele prosjektet. Samarbeidsformen gjorde utviklingsprosessen effektiv og mindre sårbar.

Litteraturliste

Android Developers. (2025, November 21) *Material Design 3 in compose.* Google:

https://developer.android.com/develop/ui/compose/designsystems/material3

Android Developers. (2025, November 24) *ViewModel Overview.* Google:

https://developer.android.com/topic/libraries/architecture/viewmodel

Android Developers. (2025, November 21) *Loading Images.* Google:

https://developer.android.com/develop/ui/compose/graphics/images/loading

Android Developers. (2025, November 17) *State and Jetpack Compose.* Google:

https://developer.android.com/develop/ui/compose/state

Android Developers. (2025, November 24) *Guide to app architecture.* Google:

https://developer.android.com/topic/architecture

Høyskolen Kristiania. (2025*) PGR208 – App Architecture & ViewModel. (forelesning 6*, PowerPoint-Presentasjon). Canvas.

Høyskolen Kristiania. (2025*) PGR208 – Navigation & Lifecycle. (forelesning 4*, PowerPoint-Presentasjon). Canvas.

Høyskolen Kristiania. (2025*) PGR208 Compose UI. (forelesning 2-3*, PowerPoint-Presentasjon). Canvas.

Høyskolen Kristiania. (2025*) PGR208 HTTP & Retrofit. (forelesning 8-9*, PowerPoint-Presentasjon). Canvas.

Høyskolen Kristiania. (2025) PGR208 – *Local Storage Extended.* (*forelesning 11, PowerPoint-Presentasjon*). Canvas.

Etisk bruk av KI

I arbeidet med denne oppgaven har vi brukt kunstig intelligens (KI), spesifikt ChatGPT og Gemini i Android Studio, som støtteverktøy. Bruken har vært i tråd med Høyskolen Kristiania sine retningslinjer for ansvarlig bruk av KI i studier (Kristiania, 2025).

KI-verktøyene har blitt brukt som støtte, spesielt ved feilsøking når vi stod fast på grunn av merge-konflikter og uventede feil i prosjektet. Dette inkluderer forslag til hvor feil kunne ligge, forklaringer på hvorfor bestemte feilmeldinger oppstod, og hjelp med å forstå alternativer i Android Studio.

Vi har ikke brukt KI til å genere ferdig løsninger eller skrive koden for oss. All implementasjon, arkitekturvalg, dataflyt og teknisk gjennomføring er utviklet selvstendig av gruppen. KI har bidratt som veiledningsverktøy, ikke som erstatning for faglig arbeid, analyse eller refleksjon.